Recommendation System

- It Capture patterns of peoples behaviour and use it to predict what else they might want or like (based on likes,ratings,history and watchtime of the content or products).
- E-commerce, OTT platforms used recommendation system highly.
- There are two types:
  1. Content Based (YouTube next video suggestion/recommendation)
  2. Collaberative Filtering (If a person Ram bought Phone from flipkart he has also bought Back Cover and Screen Guard and in previous shopping you and Ram has bought same items so this time flipkart will give u recommendation that people have bought these items together.
  - It would be either item based or similar persona based.

In [1]:
```
import pandas as pd
import numpy as np
```

In [2]:
```
data = pd.read_csv('movies.csv')
data
```

Out[2]:

	index	genres	title
0	0	Action Adventure Fantasy Science Fiction	Avatar
1	1	Adventure Fantasy Action	Pirates of the Caribbean: At World's End
2	2	Action Adventure Crime	Spectre
3	3	Action Crime Drama Thriller	The Dark Knight Rises
4	4	Action Adventure Science Fiction	John Carter
...	...	...	...
4688	4688	Foreign Thriller	Cavite
4689	4689	Action Crime Thriller	El Mariachi
4690	4690	Comedy Romance	Newlyweds
4691	4691	Comedy Drama Romance TV Movie	Signed, Sealed, Delivered
4692	4692	Documentary	My Date with Drew

4693 rows × 3 columns

In [3]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4693 entries, 0 to 4692
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype 
---  ------  --------------  ----- 
 0   index   4693 non-null   int64 
 1   genres  4666 non-null   object
 2   title   4693 non-null   object
dtypes: int64(1), object(2)
memory usage: 110.1+ KB
```

In [4]:
```
data.isnull().sum()
```

Out[4]:
```
index      0
genres    27
title      0
dtype: int64
```

In [5]:
```
data[data['genres'].isnull()]
```

Out[5]:

	index	genres	title
3883	3883	NaN	Iguana
3904	3904	NaN	Sardaarji
3978	3978	NaN	Sharkskin
4014	4014	NaN	The Book of Mormon Movie, Volume 1: The Journey
4027	4027	NaN	Hum To Mohabbat Karega
4197	4197	NaN	The Algerian
4218	4218	NaN	Crowsnest
4289	4289	NaN	Lisa Picard Is Famous
4304	4304	NaN	Sparkler
4317	4317	NaN	Childless
4358	4358	NaN	Harrison Montgomery
4403	4403	NaN	Light from the Darkroom
4447	4447	NaN	The Outrageous Sophie Tucker
4450	4450	NaN	America Is Still the Place
4459	4459	NaN	The Little Ponderosa Zoo
4463	4463	NaN	Diamond Ruff
4466	4466	NaN	Mutual Friends
4468	4468	NaN	Rise of the Entrepreneur: The Search for a Bet...
4478	4478	NaN	Gory Gory Hallelujah
4506	4506	NaN	The Big Swap
4517	4517	NaN	The Looking Glass
4528	4528	NaN	Death Calls
4552	4552	NaN	UnDivided
4569	4569	NaN	Short Cut to Nirvana: Kumbh Mela
4576	4576	NaN	Reality Show
4607	4607	NaN	An American in Hollywood
4609	4609	NaN	The Blood of My Brother: A Story of Death in Iraq

In [6]:
```
data[data['genres'].isnull()]['title'].tolist()
```

Out[6]:
```
['Iguana',
 'Sardaarji',
 'Sharkskin',
 'The Book of Mormon Movie, Volume 1: The Journey',
 'Hum To Mohabbat Karega',
 'The Algerian',
 'Crowsnest',
 'Lisa Picard Is Famous',
 'Sparkler',
 'Childless',
 'Harrison Montgomery',
 'Light from the Darkroom',
 'The Outrageous Sophie Tucker',
 'America Is Still the Place',
 'The Little Ponderosa Zoo',
 'Diamond Ruff',
 'Mutual Friends',
 'Rise of the Entrepreneur: The Search for a Better Way',
 'Gory Gory Hallelujah',
 'The Big Swap',
 'The Looking Glass',
 'Death Calls',
 'UnDivided',
 'Short Cut to Nirvana: Kumbh Mela',
 'Reality Show',
 'An American in Hollywood',
 'The Blood of My Brother: A Story of Death in Iraq']
```

In [7]:
```
genres_list = [
    "Drama, Thriller", "Comedy, Fantasy", "Drama", "Adventure, Drama", "Com
    "Thriller", "Horror", "Comedy", "Comedy, Drama", "Drama", "Drama", "Thr
    "Drama", "Family", "Crime, Drama", "Comedy, Drama", "Documentary", "Com
    "Drama", "Drama", "Horror", "Documentary", "Documentary", "Comedy", "Dr
    "Documentary"
]
```

In [8]:
```
#Replacing all null genres with the list of genres
data.loc[data['genres'].isnull(), 'genres'] = pd.Series(genres_list).values
```

In [9]:
```
data.isnull().sum()
```

Out[9]:
```
index     0
genres    0
title     0
dtype: int64
```

In [10]:
```
data['genres'].value_counts()
```

Out[10]:
```
Drama                                                 369
Comedy                                                284
Drama Romance                                         163
Comedy Romance                                        141
Comedy Drama                                          141
                                                     ...
Comedy Action Adventure Fantasy Science Fiction         1
Action Adventure Family                                 1
Adventure Family Fantasy Animation                      1
Action Adventure Mystery Thriller                       1
Comedy Drama Romance TV Movie                           1
Name: genres, Length: 1144, dtype: int64
```

In [11]:
```
#Top 10 genres
top_10_genres = data['genres'].value_counts().head(10)
top_10_genres
```

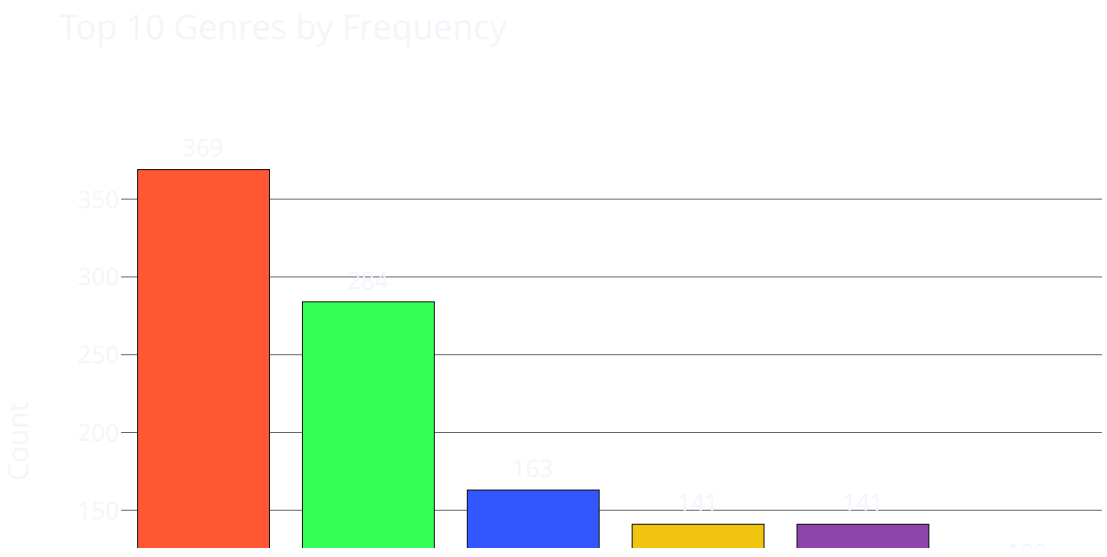
Out[11]:
```
Drama                   369
Comedy                  284
Drama Romance           163
Comedy Romance          141
Comedy Drama            141
Comedy Drama Romance    109
Horror Thriller          87
Documentary              72
Horror                   65
Drama Thriller           61
Name: genres, dtype: int64
```

In [12]:
```
import plotly.express as px
```

In [13]:
```
fig = px.bar(top_10_genres,
            x = top_10_genres.index,
            y = top_10_genres.values,
            labels = {'x':'Genres','y':'Count'},
            title = "Top 10 Genres by Frequency",
            text = top_10_genres.values)

bar_colors = ['#FF5733', '#33FF57', '#3357FF', '#F1C40F', '#8E44AD',
    '#1ABC9C', '#E74C3C', '#2ECC71', '#3498DB', '#9B59B6']
fig.update_traces(marker_color = bar_colors,texttemplate='%{text}', textpos
fig.update_layout(
    xaxis_title="Genres",
    yaxis_title="Count",
    showlegend=False,
    template="plotly_dark"
)

fig.show()
```



In [14]:
```
data.shape
```

Out[14]: (4693, 3)

In [15]:
```
data.duplicated().sum()
```

Out[15]:
```
0
```

Vectorization

(It will be disscused in detail in NLP)

- **Converting the text data to numeric vector data.**

- Single word to numeric is **Encoding**
- Group of words to numeric is **Vectorization**

Ex:

- good boy, good girl, good boy girl (vectorization) First arrange it in alphabetical order.
  - for good boy [1,0,1]
  - for good girl [0,1,1]
  - for good boy girl [1,1,1] These are known as **count vectorization**.
  - Here all words was given equally importance
- *TF-IDF Vectorization** (Term frequency-Inverse Document Frequency(count))
  - Here importance given based on occurence of the words means most weightage to the least occurence and least weightage to the frequent occurence.
- good,girl,boy (Encoding)

- So, Here we are going to use TF-IDF Vectorization because if action is repeated in most of the movies then the weightage should be given less to the action.

In [16]:
```
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer()
feature_vectors = vectorizer.fit_transform(data['genres'])
```

In [17]:
```
#we can also see the vector values
feature_vectors.toarray()
```

Out[17]:
```
array([[0.35929741, 0.41291807, 0.        , ..., 0.        , 0.        ,
        0.        ],
       [0.48161111, 0.55348558, 0.        , ..., 0.        , 0.        ,
        0.        ],
       [0.5158457 , 0.59282925, 0.        , ..., 0.        , 0.        ,
        0.        ],
       ...,
       [0.        , 0.        , 0.        , ..., 0.        , 0.        ,
        0.        ],
       [0.        , 0.        , 0.        , ..., 0.66370838, 0.        ,
        0.        ],
       [0.        , 0.        , 0.        , ..., 0.        , 0.        ,
        0.        ]])
```

In [18]:
```
# extract each genre separately
vectorizer.get_feature_names_out()
```

Out[18]:
```
array(['action', 'adventure', 'animation', 'comedy', 'crime',
       'documentary', 'drama', 'family', 'fantasy', 'fiction', 'foreign',
       'history', 'horror', 'movie', 'music', 'musical', 'mystery',
       'romance', 'science', 'thriller', 'tv', 'war', 'western'],
      dtype=object)
```

In [19]:
```
len(vectorizer.get_feature_names_out())
```

Out[19]:
```
23
```

In [20]:
```
df1 = pd.DataFrame(vectorizer.fit_transform(data['genres']).toarray(),
            columns = vectorizer.get_feature_names_out())
df1
```

Out[20]:

	action	adventure	animation	comedy	crime	documentary	drama	family	f
0	0.359297	0.412918	0.0	0.000000	0.000000	0.0	0.000000	0.0	0.
1	0.481611	0.553486	0.0	0.000000	0.000000	0.0	0.000000	0.0	0.
2	0.515846	0.592829	0.0	0.000000	0.618431	0.0	0.000000	0.0	0.
3	0.509371	0.000000	0.0	0.000000	0.610669	0.0	0.362170	0.0	0.
4	0.416822	0.479027	0.0	0.000000	0.000000	0.0	0.000000	0.0	0.
...	...	...	...	...	...	...	...	...	
4688	0.000000	0.000000	0.0	0.000000	0.000000	0.0	0.000000	0.0	0.
4689	0.546470	0.000000	0.0	0.000000	0.655145	0.0	0.000000	0.0	0.
4690	0.000000	0.000000	0.0	0.600813	0.000000	0.0	0.000000	0.0	0.
4691	0.000000	0.000000	0.0	0.183976	0.000000	0.0	0.158797	0.0	0.
4692	0.000000	0.000000	0.0	0.000000	0.000000	1.0	0.000000	0.0	0.

4693 rows × 23 columns

Here we have created a dataframe which consists of each genre as a column and from index number which movies has that genre filled it with some weightage value and if not filled it as zero.

Here everything got converted into numeric data. and here we can apply maths like correlation,similarity,statistics.

TF-IDF

TF(Term Frequency): No.of items 't' occurs in data. **IDF(Inverse Document Frequency):** It is a measure of how common or rare a item is across the entire corpus of document.

- So, the point to note is that its common to all the documents.
- If the word is common and appears in many documents, the idf value(normalized) will approach 0 or else approach 1 if its rarest.

idf(word) = 1+ln[n/df(t)]

where, ln = natural log means log to base e n = Total number of records t = term for which idf value has to be calculated df(t) = Number of records consists of given word

ex: idf(girl) in above example = 1 + ln[3/2] idf(boy) = 1 + ln[3/2] idf(good) = 1 + ln[3/3]

Why we use TF-IDF?

- Because we don't want to give same priority for every genre.

Use Count Vectorizer: When analyzing raw frequency or performing simple tasks where weighting isn't required.

Use TF-IDF: When working with similarity, clustering, or when frequent genres should have less importance.

Cosine Similarity

- Similarity b/w two records.
- cos(θ) = $\overline{a}$. $\overline{b}$/|$\overline{a}$| * |$\overline{b}$|

In [21]:
```
from sklearn.metrics.pairwise import cosine_similarity
similarity = cosine_similarity(feature_vectors)
print(similarity)
```

```
[[1.         0.74603222 0.43013193 ... 0.         0.         0.        ]
 [0.74603222 1.         0.57655946 ... 0.         0.         0.        ]
 [0.43013193 0.57655946 1.         ... 0.         0.         0.        ]
 ...
 [0.         0.         0.         ... 1.         0.30621223 0.        ]
 [0.         0.         0.         ... 0.30621223 1.         0.        ]
 [0.         0.         0.         ... 0.         0.         1.        ]]
```

Here we can observe the similarity score between movies Avatar and avatar is 1 and so on

- This is called as **similarity metrics** earlier it was similarity score.

In [22]:
```
similarity.shape
```

Out[22]:
```
(4693, 4693)
```

Creating a list with all the movies names in the dataset.

In [23]:
```
list_of_movies = data['title'].tolist()
print(list_of_movies)
```

```
rrowland', 'Big Hero 6', 'Wreck-It Ralph', 'The Polar Express', 'Indepe
ndence Day: Resurgence', 'How to Train Your Dragon', 'Terminator 3: Ris
e of the Machines', 'Guardians of the Galaxy', 'Interstellar', 'Incepti
on', 'The Hobbit: An Unexpected Journey', 'The Fast and the Furious',
'The Curious Case of Benjamin Button', 'X-Men: First Class', 'The Hunge
r Games: Mockingjay - Part 2', "The Sorcerer's Apprentice", 'Poseidon',
'Alice Through the Looking Glass', 'Shrek the Third', 'Warcraft', 'Term
inator Genisys', 'The Chronicles of Narnia: The Voyage of the Dawn Trea
der', 'Pearl Harbor', 'Transformers', 'Alexander', 'Harry Potter and th
e Order of the Phoenix', 'Harry Potter and the Goblet of Fire', 'Hancoc
k', 'I Am Legend', 'Charlie and the Chocolate Factory', 'Ratatouille',
'Batman Begins', 'Madagascar: Escape 2 Africa', 'Night at the Museum: B
attle of the Smithsonian', 'X-Men Origins: Wolverine', 'The Matrix Revo
lutions', 'Frozen', 'The Matrix Reloaded', 'Thor: The Dark World', 'Mad
Max: Fury Road', 'Angels & Demons', 'Thor', 'Bolt', 'G-Force', 'Wrath o
f the Titans', 'Dark Shadows', 'Mission: Impossible - Rogue Nation', 'T
he Wolfman', 'Bee Movie', 'Kung Fu Panda 2', 'The Last Airbender', 'Mis
sion: Impossible III', 'White House Down', 'Mars Needs Moms', 'Flushed
Away', 'Pan', 'Mr. Peabody & Sherman', 'Troy', "Madagascar 3: Europe's
Most Wanted", 'Die Another Day', 'Ghostbusters', 'Armageddon', 'Men in
```

Getting the movie name from user

In [24]:
```
movie_name = input('Enter the movie name: ')
```

```
Enter the movie name: spiderman
```

Finding the close match for the movie name given by the user

In [25]:
```
import difflib
find_close_match = difflib.get_close_matches(movie_name,
                                             list_of_movies)
print(find_close_match)
```

```
['Spider-Man', 'Superman', 'Spider-Man 3']
```

Here it is identified by nearest word

In [26]:
```
ind = int(input('Enter the movie position number: '))
close_match = find_close_match[ind]
print(close_match)
```

```
Enter the movie position number: 0
Spider-Man
```

In [27]:
```
#finding the index of the movie with title

index_of_movies = data[data['title'] == close_match]['index'].values[0]
print(index_of_movies)
```

```
157
```

In [29]:
```
#getting a list of similarity score

similarity_score = list(enumerate(similarity[index_of_movies]))
print(similarity_score)
```

```
[(0, 0.6213394711432644), (1, 0.8328587610242142), (2, 0.29829430083429
753), (3, 0.29455028806547257), (4, 0.24103244162247517), (5, 0.8328587
610242142), (6, 0.0), (7, 0.24103244162247517), (8, 0.511689679524701
9), (9, 0.8328587610242142), (10, 0.6213394711432644), (11, 0.267605810
0662027), (12, 0.8328587610242142), (13, 0.2244955558978365), (14, 0.62
13394711432644), (15, 0.5116896795247019), (16, 0.24103244162247517),
(17, 0.8328587610242142), (18, 0.2557197710016801), (19, 0.832858761024
2142), (20, 0.8328587610242142), (21, 0.3795869275012295), (22, 0.63255
42065155976), (23, 0.6325542065155976), (24, 0.3439670878676505), (25,
0.0), (26, 0.24103244162247517), (27, 0.22395349107632803), (28, 0.2239
5349107632803), (29, 0.3216496147477704), (30, 0.8328587610242142), (3
1, 0.24103244162247517), (32, 0.5116896795247019), (33, 0.2239534910763
2803), (34, 0.0), (35, 0.24103244162247517), (36, 0.24103244162247517),
(37, 0.5116896795247019), (38, 0.8328587610242142), (39, 0.241032441622
47517), (40, 0.0), (41, 0.22395349107632803), (42, 0.0), (43, 0.2500908
4599994047), (44, 0.5782626493942683), (45, 0.2099218560930189), (46,
0.6213394711432644), (47, 0.24103244162247517), (48, 0.794336750337501
2), (49, 0.0), (50, 0.7365442805050979), (51, 0.24103244162247517), (5
2, 0.24103244162247517), (53, 0.3795869275012295), (54, 0.0), (55, 0.21
```

In [30]:
```
#sorting the movies based on their similarity score

sorted_similarity_score = sorted(similarity_score, key = lambda x:x[1],
                                 reverse = True)
print(sorted_similarity_score)
```

```
[(113, 1.0), (157, 1.0), (424, 1.0), (661, 1.0), (1341, 1.0), (2068, 1.
0), (3529, 1.0), (4549, 1.0), (531, 0.9248771134364526), (146, 0.902805
7754257959), (883, 0.9028057754257959), (541, 0.8754621572322778), (79
5, 0.8754621572322778), (921, 0.8754621572322778), (1273, 0.87546215723
22778), (1997, 0.8754621572322778), (1, 0.8328587610242142), (5, 0.8328
587610242142), (9, 0.8328587610242142), (12, 0.8328587610242142), (17,
0.8328587610242142), (19, 0.8328587610242142), (20, 0.832858761024214
2), (30, 0.8328587610242142), (38, 0.8328587610242142), (70, 0.83285876
10242142), (96, 0.8328587610242142), (105, 0.8328587610242142), (124,
0.8328587610242142), (127, 0.8328587610242142), (197, 0.832858761024214
2), (204, 0.8328587610242142), (206, 0.8328587610242142), (259, 0.83285
87610242142), (312, 0.8328587610242142), (325, 0.8328587610242142), (32
6, 0.8328587610242142), (375, 0.8328587610242142), (480, 0.832858761024
2142), (706, 0.8328587610242142), (737, 0.8328587610242142), (826, 0.83
28587610242142), (1419, 0.8328587610242142), (2302, 0.832858761024214
2), (2348, 0.8328587610242142), (2448, 0.8328587610242142), (2609, 0.83
28587610242142), (208, 0.8218255540371966), (296, 0.8218255540371966),
(103, 0.8158506654501922), (159, 0.8158506654501922), (578, 0.815850665
4501922), (793, 0.8158506654501922), (2325, 0.8158506654501922), (3693,
```

In [31]:
```
data.iloc[157]
```

Out[31]:
```
index                157
genres    Fantasy Action
title         Spider-Man
Name: 157, dtype: object
```

In [32]:
```
data.iloc[113]
```

Out[32]:
```
index                 113
genres    Fantasy Action
title            Hancock
Name: 113, dtype: object
```

We can see based on full similarity of genre it has been sorted

Recommendation of movies based on similarity

In [33]:
```
print('Movies you can also watch: \n')
i = 1
for movie in sorted_similarity_score:
    index = movie[0]
    title_from_index = data[data.index==index]['title'].values[0]
    if (i<=10):
        print(i,'-',title_from_index)
    i+=1
```

```
Movies you can also watch:

1 - Hancock
2 - Spider-Man
3 - Batman Returns
4 - Elektra
5 - Batman
6 - Mortal Kombat
7 - The Beastmaster
8 - Ink
9 - Immortals
10 - Ghostbusters
```

These are the 10 recommended movies based on best similarity score of the selected movie